МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №4

по дисциплине

«Технологии компонентного программирования»

«Jython: взаимодействие Java и Python»

Выполнил:
Лебедевич Артем Владимирович
магистрант группы 556301

Преподаватель:
Герман Олег Витольдович
кандидат технических наук
доцент кафедры ИТАС

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2

2 Теоретические сведения 3
2.1 Jython 3
2.2 Установка 3
2.3 Различия при работе с JDK 17 3

3 Реализация 5
3.1 Структура программы 5
3.2 Примеры из методических материалов 5
3.3 Приложение с меню 6
3.4 Проверка без экрана 7
3.5 Окно проверки 7

4 Результаты работы программы 8

5 Ответы на контрольные вопросы 12

Заключение 13

Список использованных источников 14

1 ЦЕЛЬ РАБОТЫ

Изучить взаимодействие языков Java и Python с помощью Jython. Выполнить примеры из методических материалов и создать приложение Jython с визуальным классом Java: окном `JFrame`, меню которого загружает две разные картинки и рисует синусоиду на холсте.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Jython

Jython — реализация языка Python (версии 2.7) для виртуальной машины Java. Программа на Jython компилируется в байт-код JVM, поэтому может импортировать классы Java как обычные модули Python, вызывать их методы, перехватывать исключения Java и наследовать от классов Java. Обратное направление — запуск кода Python из программы на Java — выполняется вызовом библиотеки. Основные понятия приведены в таблице 2.1.

Таблица 2.1 – Основные понятия

Понятие	Смысл
`jython-standalone.jar`	интерпретатор и стандартная библиотека в одном файле: `java -jar jython-standalone-2.7.3.jar script.py`
`from java.io import File`	класс Java импортируется как имя Python
`class MyClass(Object)`	класс Python, наследующий класс Java; его экземпляры — объекты Java
`sys.path.append("x.jar")`	добавляет jar-файл в путь импорта
`PythonInterpreter`	класс Java, выполняющий код Python внутри программы на Java
Swing	библиотека графического интерфейса Java: `JFrame`, `JMenuBar`, `JPanel`, `paintComponent`

2.2 Установка

Jython распространяется в виде jar-файла. В работе использован `jython-standalone-2.7.3.jar` с Maven Central; контрольная сумма SHA-1 проверяется сравнением с файлом `.sha1`, который публикует Maven. Для запуска нужна JDK; использован Temurin 17. Скрипт `scripts/fetch_runtimes.py` скачивает jar в каталог `tools/`, а модуль `lab4/runtime.py` находит Java и запускает сценарии.

2.3 Различия при работе с JDK 17

Методичка рассчитана на Java 8. С JDK 17 обнаружились следующие отличия.

– Инструкции `import javax.swing as swing` и `import java.awt as awt` завершаются ошибкой `No module named swing`, хотя `from javax.swing import JFrame` работает. Начиная с Java 9 классы находятся в модулях, и сканер пакетов Jython их не перечисляет. Модуль

`jcompat.py` регистрирует нужные пакеты вызовом `sys.add_package`, после чего импорты из методички работают.

– Защищённый метод родительского класса вызывается как `self.super__paintComponent(g)`, а не `JPanel.paintComponent(self, g)`.
– Jython — это Python 2.7: для функции `print` нужен `from __future__ import print_function`, а строки с символом π задаются литералами `u` (запуск с `-Dfile.encoding=UTF-8`).
– Окно `JFrame` нельзя создать в режиме без экрана (`java.awt.headless=true`), поэтому автоматически проверяемая часть выполнена на `JPanel` и `JRootPane`.

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Листинг 3.1 – Структура лабораторной работы

```
lab4/jython/java_version.py  System.getProperty, currentTimeMillis
lab4/jython/hello_world.py   sys.stdout.write и print
lab4/jython/files_demo.py    File, FileWriter, IOException
lab4/jython/my_class.py      класс Python на основе java.lang.Object
lab4/jython/greet_frame.py   JFrame с кнопками Greet и Clear
lab4/jython/image_viewer.py  показ JPEG в компоненте Swing
lab4/jython/array_list.py    коллекция Java в цикле Python
lab4/jython/draw_image.py    рисование в BufferedImage
lab4/jython/call_java_class.py, jy10.py, music_player.py, jcompat.py
lab4/jython/menu_app.py      задание: меню, две картинки, синусоида
lab4/java/com/MyClass.java   класс Java из jar-файла
lab4/java/JythonHelloWorld.java  Java запускает Python
lab4/runtime.py              поиск Java и Jython, запуск сценариев
lab4/cases.py                проверяемые сценарии
lab4/runner.py, lab4/gui.py  вывод в консоль и окно
assets/lab4/                 две картинки: JPEG и PNG
```

3.2 Примеры из методических материалов

Все примеры выполнены. Первый пример (листинг 3.2) показывает, что класс Java `System` используется как модуль Python; следующий создаёт файл средствами `java.io` и перехватывает исключение Java `IOException`; класс `MyClass` (листинг 3.3) наследует `java.lang.Object`, поэтому его экземпляр является объектом Java: для него работают `hashCode`, `getClass` и проверка `isinstance`.

Листинг 3.2 – Класс Java из Python

```
from java.lang import System  # Java import

print('Running on Java version: ' + System.getProperty('java.version'))
print('Unix time from Java: ' + str(System.currentTimeMillis()))
```

Листинг 3.3 – Класс Python, наследующий класс Java

```
from java.lang import Object

class MyClass(Object):
    def __init__(self):
        Object.__init__(self)
        self.message = "Hello, world! Form Class"

    def printMessage(self):
        print(self.message)
```

Примеры из лекции показывают взаимодействие в обе стороны. Класс `com.MyClass` на Java компилируется и упаковывается в jar-файл, после чего подключается из Jython; программа `JythonHelloWorld.java`, наоборот, запускает интерпретатор внутри Java (листинг 3.4).

Листинг 3.4 – Java запускает Python

```
try (PythonInterpreter python = new PythonInterpreter()) {
    python.exec("print('Hello Python World!')");
    python.exec("from jy10 import greet\ngreet('John')");
    python.set("n", 10);
    python.exec("result = n ** 2");
    System.out.println(python.get("result", Integer.class));
}
```

3.3 Приложение с меню

Окно собрано из трёх частей: строка меню `File / Draw / Help`, холст и строка состояния. Меню `Draw` содержит пункты `Picture 1 (JPEG)`, `Picture 2 (PNG)`, `Sine wave` и `Clear` с комбинациями клавиш `Cmd+1…0` (на других системах `Ctrl`).

Холст — класс `Canvas`, наследующий `JPanel`; его метод `paintComponent` написан на Python (листинг 3.5). Картинка читается средствами Java (`ImageIO.read` понимает и JPEG, и PNG), масштабируется по размеру холста с сохранением пропорций и центрируется. Синусоида рисуется линией `Path2D` из одного отрезка на каждый столбец пикселей; подписаны оси, деления сетки через $\pi/2$ и метки $0, \pi, 2\pi, \ldots, 4\pi$.

Листинг 3.5 – Холст: метод paintComponent

```
class Canvas(JPanel):
    def paintComponent(self, g):
        self.super__paintComponent(g)  # let Swing paint the background first
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON)
        width, height = self.getWidth(), self.getHeight()
        if self.mode == "picture":
            self.draw_picture(g, width, height)
        elif self.mode == "sine":
            self.draw_sine(g, width, height)

    def draw_sine(self, g, width, height):
        left, top, w, h = self.sine_frame(width, height)
        x_max = 2 * math.pi * self.PERIODS
        path = Path2D.Double()
        for i in range(w + 1):
            x = x_max * i / float(w)
            if i == 0:
                path.moveTo(px(x), py(math.sin(x)))
            else:
                path.lineTo(px(x), py(math.sin(x)))
        g.setColor(BLUE)
        g.draw(path)
```

События всех пунктов меню принимает один объект `Controller`, который реализует интерфейс `ActionListener` и выбирает действие по строке команды пункта (листинг 3.6).

Листинг 3.6 – Обработка выбора пункта меню

```
class Controller(ActionListener):
    def actionPerformed(self, event):
        command = event.getActionCommand()
        if command in PICTURES:
```

```
            self.canvas.show_picture(PICTURES[command])
        elif command == "sine":
            self.canvas.show_sine()
        elif command == "clear":
            self.canvas.clear()
```

3.4 Проверка без экрана

Строка меню, холст и строка состояния устанавливаются в `JRootPane` (листинг 3.7). Такой объект принадлежит каждому `JFrame`, но существует и без окна. Благодаря этому режим `--snapshot` рисует содержимое окна в файл PNG на машине без экрана, а тесты проверяют пиксели. Режим `--popup Draw` дорисовывает в снимок раскрытое меню.

Листинг 3.7 – Сборка окна

```
def install(root, controller, canvas, status):
    root.setJMenuBar(build_menu_bar(controller))
    root.getContentPane().add(canvas, BorderLayout.CENTER)
    root.getContentPane().add(status, BorderLayout.SOUTH)

class MenuFrame(JFrame):
    def __init__(self, visible=True):
        JFrame.__init__(self, TITLE)
        self.canvas = Canvas()
        self.status = make_status()
        self.controller = Controller(self.canvas, self.status, self)
        install(self.getRootPane(), self.controller, self.canvas, self.status)
        self.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
        self.pack()
        self.setVisible(visible)
```

3.5 Окно проверки

Окно `python main.py --lab 4 --gui` написано на wxPython: три кнопки слева показывают снимок содержимого окна для выбранного вида, три кнопки справа запускают настоящие окна Swing отдельными процессами JVM, справа расположен список из четырнадцати сценариев.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

На рисунке 4.1 показано окно проверки после выполнения всех сценариев (14 из 14); на снимке раскрыто меню `Draw`.

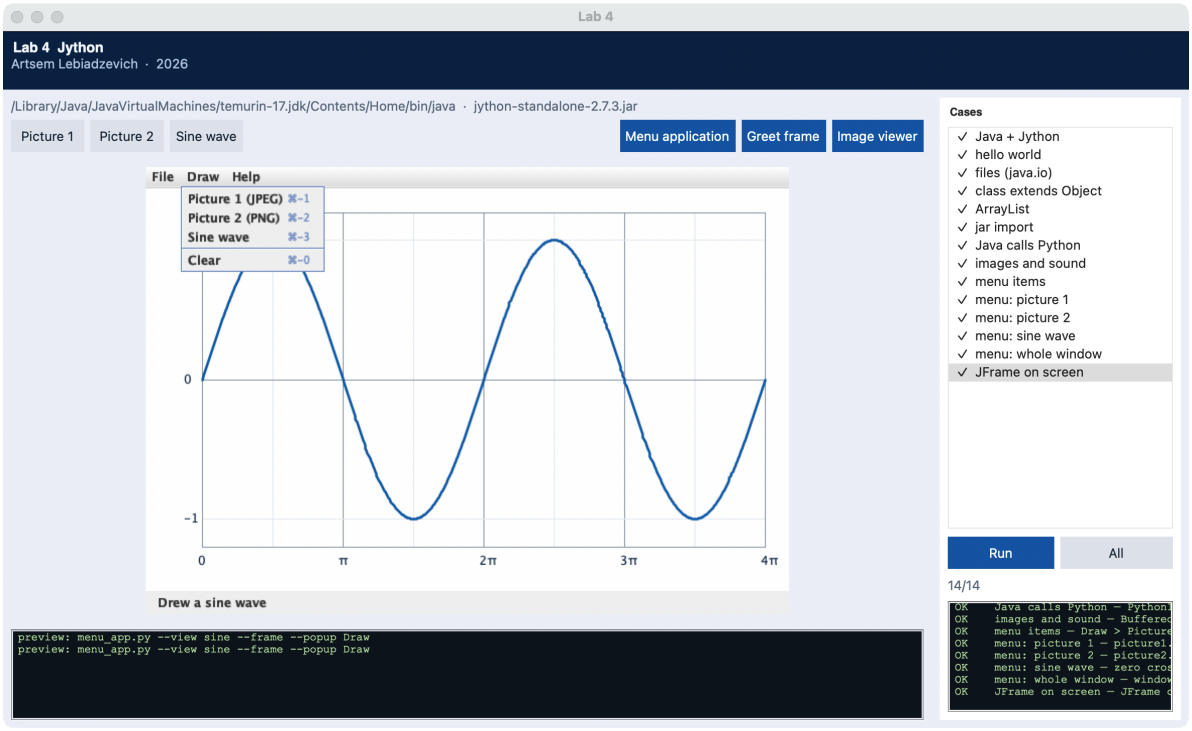


Рисунок 4.1 – Окно лабораторной работы: снимок приложения и список проверок

Содержимое окна приложения для каждого пункта меню показано на рисунках 4.2–4.5.

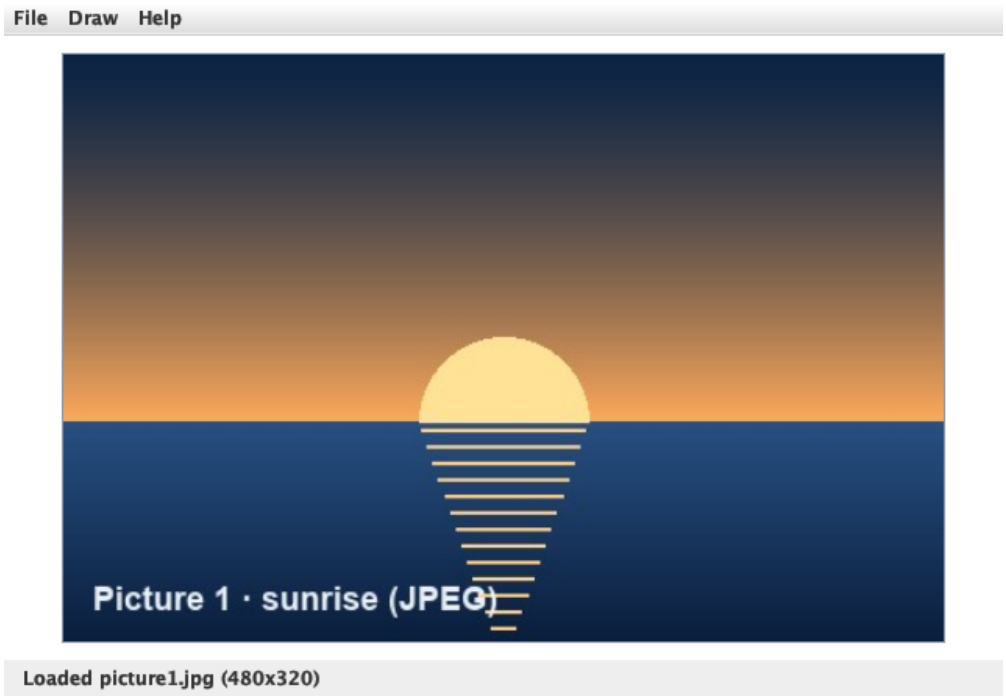


Рисунок 4.2 – Draw → Picture 1: файл JPEG

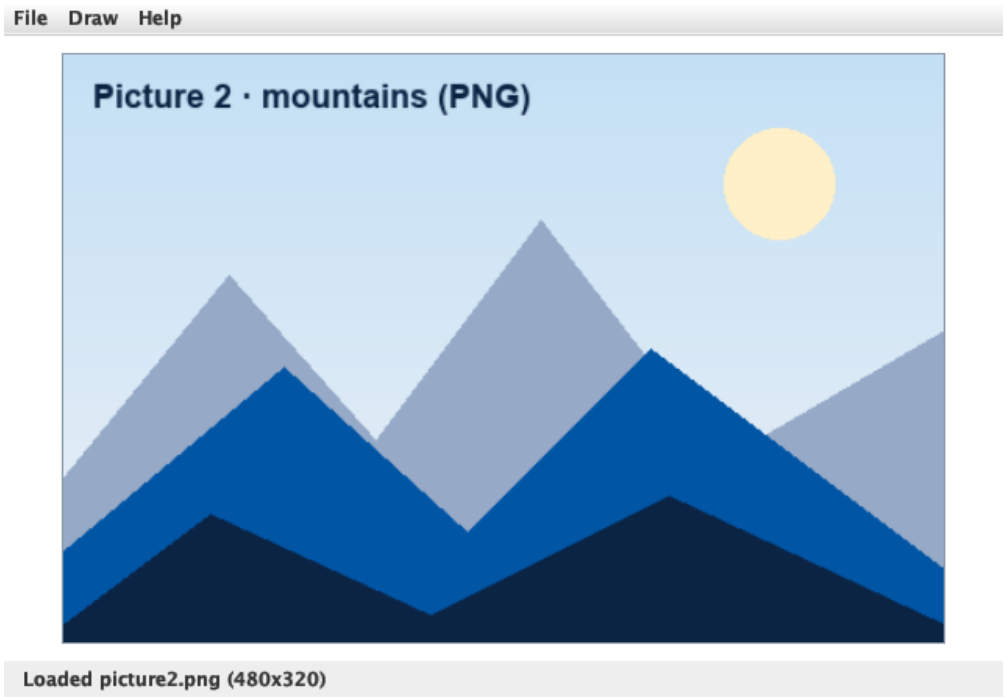


Рисунок 4.3 – Draw → Picture 2: файл PNG

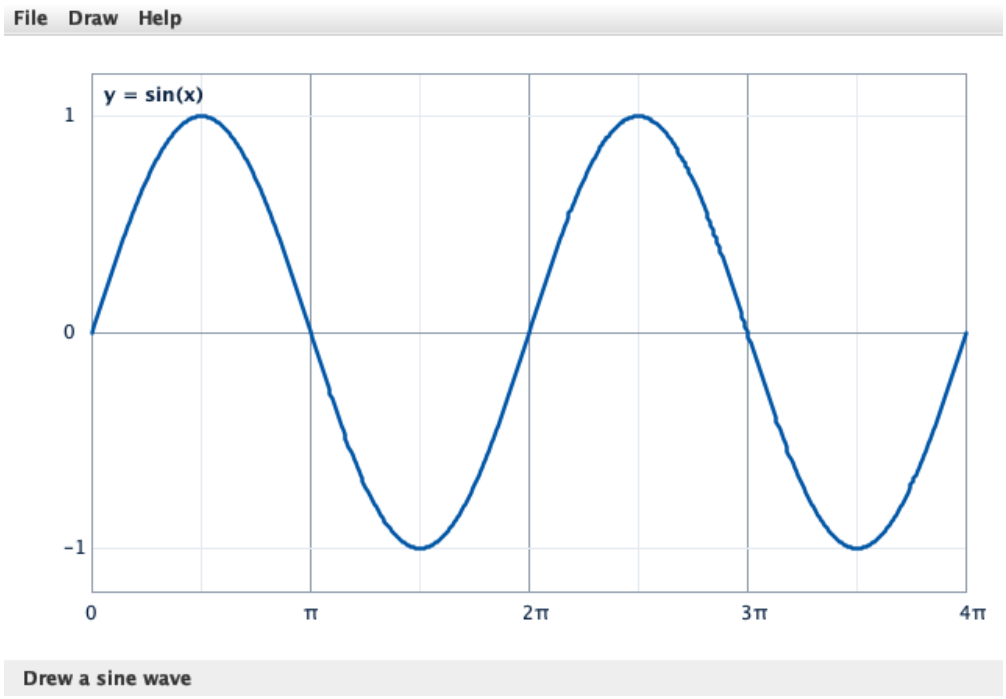


Рисунок 4.4 – Draw → Sine wave: $y = \sin x$ на отрезке $[0, 4\pi]$

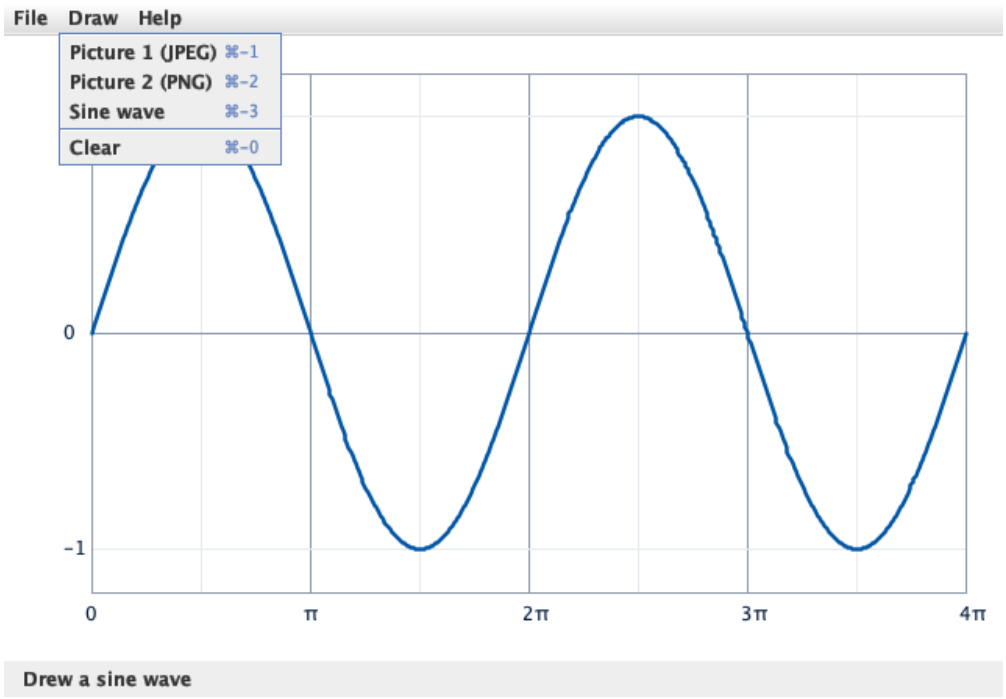


Рисунок 4.5 – Раскрытое меню Draw с сочетаниями клавиш

Команда `python main.py --lab 4` выполняет сценарии без окна; вывод приведён в листинге 4.1. Результат проверяется по пикселям (таблица 4.1): сравнение с исходным файлом, а не только отсутствие ошибки.

Листинг 4.1 – Вывод программы

```
environment -> Java 17.0.17 + Jython 2.7.3 (jython-standalone-2.7.3.jar)
hello_world -> sys.stdout.write and print give the same result
files -> java.io.File and FileWriter created example_file.txt (24 bytes)
```

```
python_class_extends_java -> class MyClass(Object): a Python class is a Java object
java_collections -> ArrayList is iterated with a Python for loop and has len()
jar_import -> sys.path.append(myclass.jar); from com import MyClass -> Hello, dear oleg
java_calls_python -> PythonInterpreter inside Java: exec, import of jy10, set/get of values
images_and_sound -> BufferedImage rect and oval drawn, JPEG read by ImageIO (480x320), WAV opened by javax.sound
menu_items -> Draw > Picture 1, Picture 2, Sine wave, Clear: the canvas follows every click
menu_picture1 -> picture1.jpg drawn at 564x376, mean error 0.6/255
menu_picture2 -> picture2.png drawn at 564x376, mean error 0.4/255
menu_sine -> zero crossings at 197, 336, 476 px, crest at 124 px, amplitude 0.84 of the half-height
menu_window -> window content 640x444 (menu bar, canvas, status line); the open menu changes (35, 19, 178, 104)
swing_window -> JFrame opened, Greet printed the name, Clear emptied the field
```

Таблица 4.1 – Проверяемые сценарии

Сценарий	Что проверяется
environment	версии Java и Jython, время из Java близко к времени компьютера
hello world, files	вывод двумя способами; файл создан средствами `java.io`, содержимое прочитано из Python
class extends Object	экземпляр класса Python — объект Java
ArrayList, jar import	коллекция Java в цикле Python; класс из jar-файла после `sys.path.append`
Java calls Python	`PythonInterpreter`: `exec`, импорт модуля, `set` и `get`
images and sound	в `BufferedImage` красный прямоугольник в нужной точке; JPEG размером 480×320; формат WAV
menu items	нажатие четырёх пунктов меню меняет состояние холста
picture 1, picture 2	холст совпадает с файлом, масштабированным в тот же прямоугольник (ошибка меньше 1 из 255)
sine wave	кривая непрерывна, нули в точках $\pi, 2\pi, 3\pi$ (± 3 пикселя), первый максимум в $\pi/2$, амплитуда $1/1{,}2$ высоты
whole window	размер окна больше холста, раскрытое меню меняет снимок
JFrame on screen	настоящее окно открыто, `Greet` печатает имя, `Clear` очищает поле

Автоматические тесты (`python -m unittest tests.test_labs`) повторяют основные проверки и пропускаются, если не найдены Java или jar-файл Jython. Все тесты проходят.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое Jython и как его запускают? Реализация Python 2.7 для виртуальной машины Java. Запуск: `java -jar jython-standalone-2.7.3.jar script.py` либо установленным лаунчером `jython`.

2. Как использовать классы Java в программе Jython? Импортировать как обычные имена (`from java.io import File`) и вызывать их методы. Исключения Java перехватываются инструкцией `except`, от классов Java можно наследовать (`class MyClass(Object)`), а коллекции Java допускают цикл `for` и `len`.

3. Как подключить классы из jar-файла? Добавить файл в путь импорта (`sys.path.append("x.jar")`) или указать `-classpath`, затем `from com import MyClass`.

4. Как вызвать Python из Java? Добавить jar-файл Jython в путь классов и использовать `PythonInterpreter`: `exec` выполняет код, `set` и `get` передают значения.

5. Объясните решение лабораторной работы. Класс `Canvas` — панель Swing, метод `paintComponent` которой рисует либо масштабированную картинку, либо оси и синусоиду. Пункты меню используют общий `ActionListener`, который по команде пункта переключает холст. Меню и холст собраны в `JRootPane`, поэтому их можно нарисовать в файл и проверить без экрана.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучено взаимодействие Java и Python через Jython. Выполнены все примеры методических материалов и лекции: классы Java используются в Python, класс Python наследует класс Java, jar-файл подключается к программе, а программа на Java запускает код Python. Создано приложение с окном `JFrame`, меню которого загружает две разные картинки (JPEG и PNG) и рисует синусоиду на холсте.

Практический вывод: Jython делает классы Java обычными именами Python, но привязывает программу к Python 2.7 и к версии JDK. Для JDK 17 импорты из методички потребовали обходного пути (`sys.add_package`). Размещение рисования в `JPanel`, а меню — в `JRootPane` позволило проверить графическое приложение по пикселям, не открывая окна.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Герман, О. В. Jython: лекция и лабораторная работа по дисциплине «Технологии компонентного программирования» / О. В. Герман. — Минск: БГУИР, 2024.
2. Jython Documentation [Электронный ресурс]. — Режим доступа: https://www.jython.org/.
3. Pedroni, S. Jython Essentials / S. Pedroni, N. Rappin. — Sebastopol: O'Reilly, 2002. — 296 p.
4. The Java Tutorials: Creating a GUI With Swing [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/javase/tutorial/uiswing/.